

Lab 5: Attacking Vibecoded Websites

Cyber Security: Defence against the Dark Arts

Erik Törnudd, Nikola Schuppan-Cruz — 30 September 2026

We would like to be part of the paper on the security of AI generated websites (and listed as co-authors).

Executive Summary

In this report we are presenting the result of using two different LLMs to attack vibecoded websites. The LLMs models we used were the free version of Deepseek and Claude sonnet 5 with a pro subscription. The process used was to ask both models first to list the tools needed to do a security audit on a website and then ask them to produce a script that automated running these tools. Here we discovered differences in what type of scripts the LLMs were comfortable delivering without more context where Claude refused until it was told it was for a cyber security course and

Generating Security Audit Scripts

Using the LLMs “DeepSeek” and “Claude” we generated security auditing scripts that would use all the tools we have gotten to know in the course, and more, to attack the websites. The scripts were then run against the websites and the results were recorded.

The initial two prompts for both DeepSeek and Claude were the same:

1. I am performing a security audit as part of an introductory course on cyber security. Could you please provide me with a list of commands that could be used to evaluate the security of a hosted website? Please respond in english
2. Please put all of these into an executable python script to automate this security audit on any given website

Their responses to the first prompt were the exact same, word for word. They specified the “gold standard” tools for security auditing, separated into four phases:

1. Reconnaissance: `whois`, `dig ANY`, `nslookup`, `dig axfr`, `nmap -sV -p`, `curl -I`, `whatweb`, `theHarvester -d`
2. Directory and Content Discovery: `gobuster dir -u -w`, `ffuf -u /FUZZ -w`
3. Vulnerability Scanning: `nikto -h`, `nmap --script vuln`, `testssl.sh`, `sslyze`
4. Manual Checks with `curl` like `curl /nonexistent` or `curl -X OPTIONS -i` and manual inspection of the `Set-Cookie` header and security flags like `Secure`, `HttpOnly`, and `SameSite`.

After the second prompt their responses differed, so our prompts did as well. While DeepSeek produced a script immediately, Claude’s guardrails initially prevented it from doing so.

Using DeepSeek

When presented with the task of generating a security auditing script, DeepSeek produced the python script immediately. It was bare-boned, functional and produced results.

There were no guardrails to circumvent to get the script, nor were there any guardrails encountered during the entire process of working with this DeepSeek model.

Over the course of eight sequential prompts, DeepSeek produced a script that was able to run all the tools it had listed before, and produced readable results for all scanned websites. The main workflow for this process was to run the script on select IPs, and feed the output back into DeepSeek, asking it to tailor the script to the given website, and make the output more readable. Doing this, DeepSeek added things like cookie-flag checks, “dangerous HTTP methods” checks, sensitive file checks, severity classifications for notable findings (Critical, Medium, Low, Info), advanced CLI flags, HTML reporting, and several safety fixes.

The final version (v8) featured TCP port scanning, service and banner grabbing, DNS resolution, HTML content analysis, and timing metrics for overall execution and individual checks.

Using Claude

Chat function

When presented with the task of generating a security audit script Claude was very sceptical at first, not wanting to make a general purpose tool that would fire all the different tools against any website whose IP was fed into it. After assuring it that the target websites were lab VMs, it did not hesitate to create the script, even if it did add a couple of inconvenient details in running the script, like having to type in a string confirming I have permission to run the script against the target. The script in question was running perfectly fine, in the sense that it ran all the tools available on the machine, and simply marking the other tools not found.

This script was mainly tested on my own site I had created for Lab 4 and thus verified against the manual tests I had done on the site on my own.

Claude code

To test the capabilities and differences between Claude Code and the ordinary chat function’s coding ability I asked Claude Code to create a new script testing more different tools, while also looping through all the IP addresses of the different lab servers ensuring it hit all the different websites that were created in the course, and give us all the potential weaknesses found. While the script provided was useful, I can’t say I noticed any particularly great differences in the speed or quality when using Claude Code compared to Claude’s normal chat function. This might, in part, be due to me giving it the original script for scanning a single website as inspiration for the new script. If I were to do this again, I would probably let it create something on it’s own, to see if more clear differences in quality and scope would materialize.

Attacking the sites

The full list of IPs used for this is:

```
130.208.246.171 130.208.246.173 130.208.246.176 130.208.246.177 130.208.246.180
130.208.246.168 130.208.246.166 130.208.246.170 130.208.246.164 130.208.246.175
130.208.246.174 130.208.246.167 130.208.246.165 130.208.246.185 130.208.246.213
```

For the script made with DeepSeek the main IP it was tested on is **130.208.246.173**. The script made with Claude was tested using a batching script that looped over all IPs listed above, testing all ports individually.

Batching Results DeepSeek

The following is the summarized output of the DeepSeek batch script. It shows how many potential vulnerabilities were found for each of the IPs, which ports those IPs are hosting websites on, and how long it took to run the security audit script on each IP.

```
=====
BATCH SECURITY AUDIT SUMMARY
Started : 2026-09-29 11:32:00
Finished: 2026-09-29 12:24:42
Duration: 52m 41s
Targets : 15
=====
```

Target	Status	Time	High	Med	Low	Info	Open ports
130.208.246.171	ok	6m 28s	0	8	19	90	80,8080
130.208.246.173	ok	3m 23s	0	8	11	42	22,80
130.208.246.176	ok	1m 41s	0	8	6	34	80
130.208.246.177	ok	2m 26s	0	3	7	65	80
130.208.246.180	ok	7m 40s	0	6	15	159	5000,8080
130.208.246.168	ok	7m 32s	88	222	11	6696	8000,8080
130.208.246.166	ok	16m 54s	4	6	27	4986	80,443,8080
130.208.246.170	ok	21.97s	0	0	0	1	
130.208.246.164	ok	21.48s	0	0	0	1	
130.208.246.175	ok	2.28s	0	0	0	1	
130.208.246.174	ok	2m 28s	0	12	9	36	80
130.208.246.167	ok	2m 14s	0	2	7	164	80
130.208.246.165	ok	21.18s	0	0	0	1	
130.208.246.185	ok	21.21s	0	0	0	1	
130.208.246.213	ok	23.09s	0	0	0	2	22

Manual Testing based on DeepSeek results

Using the data I got with this security audit script, I targeted **130.208.246.173** (“Rare Minecraft Worlds”), **130.208.246.177:8000** (“Smash & Rally Badminton Club”), and **130.208.246.168** (“MeowsageBoard”).

During the lab I used **130.208.246.173** for testing and developing the script. While doing this, I also attempted several manual scans and attacks, like Stored Cross-Site Scripting in the form of an html script formatted as plain-text, an .html file, and as a .png file. I also attempted SQL Injection on the login form and tried changing cookie-IDs, user-IDs, and more, through the URL and the browser’s terminal.

While it doesn't now, at the time of being targeted 130.208.246.177 was shown to have several High severity findings as well.

Despite the scan on 130.208.246.168 finding multiple severe vulnerabilities, I didn't continue manually attacking it, due to it seeming to not be set up fully. For example, the login and registration pages never loaded.

Batching Results Claude

The following is the summary output of the Claude batch script. It shows how many potential vulnerabilities were found at 11:06 29/9/26 for each of the IPs.

```
=====
CLAUDE BATCH REVIEW SUMMARY
Started : 2026-09-29 10:23:15
Finished: 2026-09-29 10:58:20
Duration: 35m 4.37s
Targets : 15
=====

Target          Status    Leads  Open ports
-----
130.208.246.164  no_site   -
130.208.246.165  no_site   -
130.208.246.166  live      11  443,8080
130.208.246.167  live      5   80
130.208.246.168  no_site   -
130.208.246.170  live      3   80
130.208.246.171  live     11  80,8080
130.208.246.173  live      5   80
130.208.246.174  live      6   80
130.208.246.175  no_site   -
130.208.246.176  live      5   80
130.208.246.177  live     10  80,8000
130.208.246.180  no_site   -
130.208.246.185  no_site   -
130.208.246.213  no_site   -
=====
Live: 8    No site: 7
=====
```

Manual Testing based on Claude results

I continued to focus on 130.208.246.166 for manual testing as it has among the most amounts of potential vulnerabilities to explore.

I used Claude again to analyse the results of the scans to help determine the following course of action in the attacks.

I made my focus the site at port 8080 ("One Piece Nakama"). The first thing I did was make an account. I tried naming it `admin` but that name was taken so I used the username `root` instead. I tried making a forum post and discovered that functionality was broken. I then proceeded to go to the command line and do a series of queries where I looked around to see what I could discover. For the most part this lead nowhere, I did find a place where there was an `id=1` placed where I tried to see what happened if I put in different values there. Unfortunately it did not lead anywhere. (The full list of commands is contained in the appendix)

Results

In spite of all our attempts, we did not manage to gain access to any accounts we hadn't previously created, we did not manage to gain access to the hosting machine of any websites, let alone root access, and all attempts to run malicious or unwanted scripts failed, even though some scripts and malicious files managed to be planted.

Appendix

For the full scripts and prompt files, visit <https://github.com/eriktornudd/CSDA-Lab5>.